

MTG Pack Scanner

Full build instructions
Version 1.0

One tabletop photograph. Independent card recognition.
Qualified matches added automatically.
Uncertain results reviewed, never concealed.

15

Primary card-count
benchmark target

>92%

Calibrated confidence
plus eligibility gates

45

Acceptance scenarios
for the application

Prepared for Addison | September 17, 2026

Developer, agency, and coding-agent handoff. Targets require real-device and real-photo testing. This package is a specification, not a completed scanner or a claim of measured recognition accuracy. Working name, not a cleared brand.

Contents

The Markdown edition is the authoritative editable specification. Use Section 22 to brief a developer or coding agent.

1. Requirements and product boundaries.....	3
2. Decisions, assumptions, and open approvals.....	4
3. User experience and screens	6
4. Camera support and realistic image budgets.....	7
5. Architecture and stack	9
6. Card catalog, reference images, and provider use	10
7. Recognition pipeline	12
8. Confidence, eligibility, and the 92% rule	14
9. Physical copies, rescans, and duplicate protection	15
10. Durable background jobs and state machines	16
11. Data model and integrity requirements	18
12. API contracts	20
13. Collection import, export, and interoperability	22
14. Dataset, model evaluation, and calibration.....	23
15. Acceptance test matrix	25
16. Security, privacy, and rights.....	27
17. Performance and cost budgets.....	28
18. Observability, administration, and recovery.....	28
19. Repository and developer deliverables	29
20. Build sequence and milestone gates	31
21. Risk register and practical mitigations	32
22. Developer/coding-agent handoff prompt.....	33
23. Definition of done and owner handoff	33
24. Source register and verification notes.....	34
25. Contents of this handoff package.....	35

Product goal: Photograph the cards from an opened Magic: The Gathering pack laid out on a table. Detect and identify each physical card independently. Process the uploaded photo on a server. Automatically add eligible matches with calibrated exact-match confidence strictly greater than 92%. Present everything else that might be a card for review, correction, or a closer photo. Preserve quantities, expose uncertainty, and make every addition reversible.

Read this first

Build a trustworthy scanner before building a broad deck-management platform. The distinguishing feature is an entire pack processed from one photograph, with minimal correction effort and no silent collection corruption.

The earlier discussion proposed 15 cards as the everyday target, approximately 20-24 as a possible upper range, and larger scans on better cameras. These are **engineering hypotheses, not validated capabilities**. Similarly, an OCR score, image similarity, or hand-weighted average is not a calibrated probability of a correct printing. This specification replaces those assumptions with measurable release gates.

Authority: Explicit requirements in this document govern implementation. Proposed defaults may be revised through a documented architecture decision. Accuracy claims, pricing claims, supported device limits, and automatic-add behavior must not be loosened silently to make a demonstration look successful.

1. Requirements and product boundaries

1.1 Core requirements

ID	Requirement	Verification
R01	Mobile-first web app with installable PWA behavior; target phones introduced in approximately 2022 or later.	Physical-device matrix and capture tests.
R02	One image containing multiple face-up cards; primary benchmark is 15 cards.	End-to-end photographed-pack evaluation.
R03	Evaluate 16-24 cards as an extended mode, not a guaranteed baseline.	Separate count-bucket qualification.
R04	Eligible results with calibrated exact-match probability greater than 0.92 can auto-add.	Policy tests and locked evaluation.
R05	Ambiguous cards receive per-card review without blocking unrelated, eligible matches.	Mixed-result integration tests.
R06	Separate card identity, printing/language, finish, and condition.	Schema, UI, and recognition tests.
R07	Accepted uploads continue processing on the server when the user leaves the page.	Close-browser and worker-restart tests.
R08	Detect quality problems and allow correction of missed, merged, or false card regions.	Detection and recovery tests.
R09	Retries and races cannot create duplicate inventory additions.	Transaction and concurrency tests.
R10	Two physical copies of the same printing remain two copies.	Repeated-card image tests.

ID	Requirement	Verification
R11	Provide collection/binder management, batch history, corrections, and CSV transfer.	Functional acceptance tests.
R12	Keep uploaded photos private and enforce retention and deletion.	Security and lifecycle tests.
R13	Auto-add stays disabled until the applicable model and use case are qualified.	Production feature-flag and registry tests.
R14	Explain failures and preserve partial results; never fabricate a match to fill a pack.	Failure-injection and unknown-card tests.

1.2 Version 1 scope

Ship sign-in for private collections; public card search where allowed by provider terms; camera capture and image upload; optional pack/set hints; card-region detection; hybrid recognition; background jobs; partial results; manual review; inventory; binders; batch undo; timestamped price estimates; CSV import/export; account deletion; administration; and operational monitoring.

English printed cards are the initial recognition qualification target. The database must support multiple languages and card faces from the beginning. Non-English, older, or unusual cards may be recognized, but unqualified categories must enter review rather than inherit an unsupported accuracy claim.

Tokens can be stored when cataloged and deliberately enabled. Ads, card backs, accessories, and art cards must not be misfiled as ordinary playable cards. Recognition support must be stated by category, not just by total catalog size.

1.3 Explicitly deferred

Do not make these prerequisites for Version 1: a full Commander deck builder, trade marketplace, payment subscriptions, native app-store releases, offline full-catalog recognition, automatic grading, counterfeit detection, automatic foil/treatment classification, overlapping-photo stitching, 40-card guarantees, video-based continuous scanning, or automatic identification of serialized numbers.

Keep interfaces extensible, but do not implement speculative infrastructure for all of these features before the pack scanner works.

1.4 Success definition

A user photographs a supported pack-sized layout, sees all detected regions, gets correct collection additions for qualified matches, resolves remaining uncertainty quickly, and can reverse or correct the batch without losing existing collection data. A system that confidently chooses the wrong printing is worse than one that asks a short, useful question.

2. Decisions, assumptions, and open approvals

Topic	Starting decision	Status
Application form	Responsive PWA plus server-side recognition.	Proposed architecture.
Primary card count	15 cards per photograph.	Benchmark target, unproven.
Extended count	16-24 cards only when that mode and photo quality qualify.	Experimental.

Topic	Starting decision	Status
First-release image limit	24 candidate card regions; overflow asks the user to split the layout.	Product limit, not a camera law.
Auto-add comparison	<code>p_exact > 0.92</code> , with additional eligibility gates.	Required behavior.
Exactly 92%	Review, not auto-add.	Explicit boundary decision.
Recognition	Artwork/full-card retrieval plus OCR and metadata checks.	Proposed architecture.
Finish default	Unknown, unless the user explicitly chooses a supported finish.	Safety default.
Condition default	Ungraded; never assume Near Mint because the pack was just opened.	Safety default.
Inference location	Server for the primary pipeline; lightweight client guidance only.	Proposed architecture.
Hosting	Container-compatible deployment; provider not selected.	Owner/developer approval in M0.
Branding and domain	Placeholder name only.	Owner approval.
Monetization	Not implemented in Version 1.	Rights and product review required first.

Support is determined by the image actually received and the evaluated browser/device path. A newer phone can still deliver a small preview frame. An older phone can deliver a useful still image. Do not infer scan capability from a camera's advertised megapixel count.

2.1 Source-backed constraints

Browser camera access through `getUserMedia()` requires a secure context and permission.

`ImageCapture.takePhoto()` can return a still-photo Blob, but requested dimensions are not a guarantee of the returned dimensions. Feature-detect and inspect the output. [\[S01\]](#)[\[S02\]](#)

Browser workers are not permanent background services; a browser may terminate service-worker work. Consequently, persistent recognition belongs on the server after upload acceptance. [\[S03\]](#)

A calibrated probability must be evaluated against observed outcomes. The raw output of a classifier or retrieval system does not automatically have this interpretation. [\[S07\]](#)[\[S08\]](#)

Pack contents are not always restricted to the main set code. For example, Wizards' Aetherdrift collecting guide describes DFT and SPG cards within its Play Boosters. Use pack information as a hint, not an unconditional set lock. [\[S09\]](#)

Source availability note: Several Scryfall documentation pages rejected direct retrieval during preparation. Relevant primary-domain search extracts were available for API rules, card fields, and price disclaimers. The developer must verify the complete current API, bulk-data, image-use, and commercial-use documentation before implementing ingestion or public release. This is a named M0 gate, not an assertion that all current terms were reviewed.

3. User experience and screens

3.1 Navigation

Provide five primary destinations: **Scan**, **Collection**, **Batches**, **Review**, and **Settings**. Public search may be available without sign-in; private collection access requires authentication.

3.2 Start scan

The user selects a destination binder or uses their saved default. Show the automatic-add setting and threshold before the first automatic scan and whenever the user changes that setting.

Optional inputs: pack/product hint, set hint, expected physical item count, language hint, and an explicitly chosen finish default. Expected count must be optional. Never hard-code that every pack contains 15 playable cards.

Offer **Take photo** and **Choose photo**. Suggested preparation text:

Lay cards face-up with visible space between them. Keep every edge inside the picture. Use even lighting and avoid reflections. Start with about 15 cards; use fewer when the app requests a closer photo.

Do not require a fixed inch measurement between cards. Separation must be sufficient for reliable detection in the captured image.

3.3 Capture and upload

Show capture guidance when supported: detected regions, likely blur or glare, clipped edges, and estimated text readability. Treat preview guidance as provisional because the final photo can differ from the preview.

Distinguish these states clearly:

- **Photo on this device:** not yet safe to leave.
- **Uploading:** bytes are still transferring.
- **Uploaded and accepted:** image is durably stored and processing is scheduled.
- **Processing:** server is working; leaving the page is safe.

Do not display “processing in background” before server acceptance.

3.4 Processing and partial results

Show the original image with numbered, selectable regions. Display actual stage progress and processed counts, not a decorative percentage that implies measurements the system does not have.

Example interface copy, using fictional counts:

15 regions detected. 11 cards added. 2 matches need review. 1 region needs a closer photo. 1 insert excluded.

Each region needs both an icon/text label and an optional color. Color alone must not convey status.

3.5 Review one card

Present the user's crop beside the reference candidate. Show the name, set, collector number, language, relevant treatment details, and finish status. Provide up to three useful candidate choices, **Search all printings**, **Take closer photo**, and **Not a card / Ignore**.

Explain the reason: “Card name identified; printing is uncertain,” “Bottom text unreadable,” “Set and image disagree,” or “This layout is not yet qualified for automatic addition.” Do not show an unexplained percentage as the only reason for review.

Allow users to edit the crop or draw a missed region. A close-up must target the existing observation so it cannot increment quantity a second time.

3.6 Completion

Show: detected objects, physical card observations, added copies, pending decisions, excluded items, unknown finishes, ungraded copies, and any expected-count mismatch. Distinguish **processing complete** from **all decisions resolved**.

The batch is complete when processing has ended and every remaining observation is accepted, intentionally ignored, or retained as unresolved with the user's explicit choice. Finishing a batch must not silently add unresolved entries.

3.7 Collection and batch actions

Users can search and filter by card, set, binder, finish, language, and unresolved metadata. Show quantities of physical copies, not just unique names. Support move, correct, remove, and undo. A printing correction should replace the attribution of the original copy rather than create another copy.

Batch history must remain usable after photo retention expires. Show “Photo expired under retention policy” rather than a broken image. Inventory and its provenance can survive photo deletion unless the user deletes the account or collection data.

3.8 Accessibility and usability

Support keyboard and screen-reader operation, sensible focus movement during review, sufficiently large touch targets, zoomable crop comparisons, reduced motion, and non-color status labels. Preserve in-progress review after refresh. Explain permissions in plain language and give a file-upload fallback when live camera access fails.

4. Camera support and realistic image budgets

4.1 Qualification baseline

Use an actual 12-megapixel still image as a starting benchmark, not as a minimum marketing requirement. Apple documents a 12 MP main camera for the iPhone 14, making that device one useful older-phone test case. It does not establish what every browser capture path will deliver. [\[S10\]](#)

Test at least: one 2022-era iPhone; one 2022-era Android flagship; one lower-cost Android phone; one newer iPhone; and one newer Android phone. Record the exact model, OS build, browser version, capture method, image dimensions, and image format. Add older or other devices only after testing.

4.2 Illustrative geometry, not measured recognition performance

For upright cards with an assumed width-to-height ratio of 5:7, a 4032 x 3024 landscape image, 90% usable span in each direction, and gaps equal to 5% of a card dimension:

```
h = min(
    0.90 * image_height / (rows + 0.05 * (rows - 1)),
    0.90 * image_width /
        ((5 / 7) * (columns + 0.05 * (columns - 1)))
)
card_width = h * (5 / 7)
```

Cards	Layout	Approximate card crop	Meaning
6	3 x 2	948 x 1328 px	More source detail per card.
12	4 x 3	627 x 878 px	Useful benchmark layout.
15	5 x 3	627 x 878 px	Primary product benchmark.
20	5 x 4	468 x 656 px	Less footer-text detail.
24	6 x 4	468 x 656 px	Extended-mode experiment.
30	6 x 5	374 x 523 px	Outside Version 1 product limit.

These are geometric upper-budget examples before blur, lens effects, perspective, compression, and imperfect framing. They do not demonstrate that collector-number OCR will work. Layout affects which image dimension is limiting, so simply dividing image height by row count is insufficient.

4.3 Capture strategy

Implement feature detection rather than user-agent promises. Try a rear-facing camera stream for preview. Where supported and validated, use still capture. Also provide a native camera/file-input route, with the HTML capture attribute treated as a hint rather than a universal guarantee. [\[S01\]](#)[\[S02\]](#)[\[S11\]](#)

Inspect returned width, height, orientation, file size, and decode success. Record the capture path. Do not upscale a 1920 x 1080 preview to a large canvas and describe it as a high-resolution photograph. A canvas snapshot is a fallback, and its actual quality may require fewer cards.

Do not require browser access to full advertised 48/50 MP output. Stop camera tracks when leaving capture. Correctly handle EXIF rotation and the transformation between displayed coordinates and source-image coordinates.

4.4 Image-size safety

Initial configurable limits: **35 MB upload, 60 million decoded pixels, 24 candidate card regions per Version 1 scan**. These are protective starting settings and must be load-tested. Accept JPEG, PNG, and WebP. Support HEIC/HEIF only through a tested, maintained, licensed decoding path; otherwise offer clear guidance to select a supported image.

Avoid decoding several full-resolution copies on the phone. An uncompressed 48 MP RGBA buffer alone contains approximately 192 million bytes before overhead. Upload the source Blob directly where practical; generate a small preview separately.

4.5 Dynamic quality guidance

Measure crop geometry, blur, clipping, occlusion, glare over useful regions, and estimated character scale. Retain native crop dimensions separately from any resized inference tensor. Upsampling must not increase the recorded amount of source detail.

Learn actionable quality cutoffs from the benchmark. Do not ship universal rules such as “800 pixels guarantees exact printing.” If the footer is unreadable but artwork is clear, identify the name provisionally and request printing confirmation when needed.

5. Architecture and stack

5.1 Recommended starting stack

Layer	Default choice	Responsibility
Frontend	TypeScript, React, Next.js	Mobile UI, capture, review, collection, PWA shell.
Backend API	Python, FastAPI, Pydantic	Authenticated contracts, jobs, catalog, inventory.
Database	PostgreSQL with pgvector	Durable records, catalog search, vector retrieval.
Background execution	Celery with Redis broker	Dispatch and execution of recognition work.
Image processing	OpenCV and a maintained image decoder	Regions, geometry, crop preparation, image checks.
OCR	Tesseract baseline behind a replaceable interface	Name and footer extraction; benchmark alternatives.
Image matching	Versioned embedding model via ONNX Runtime	Artwork and full-card retrieval.
Object storage	Private S3-compatible storage	Uploaded images, crops, model artifacts.
Authentication	Maintained OIDC provider and server-verified sessions	Identity, login, recovery; no custom password system.
Testing	Pytest, frontend tests, Playwright, device checks	Policy, API, browser, and end-to-end validation.

These are implementation choices, not proof that any untrained model recognizes MTG cards adequately. ONNX Runtime provides an inference runtime; pgvector provides vector-search facilities. Neither supplies the finished recognizer. [S12][S13]

Select currently supported language/runtime and package versions at kickoff. Pin exact versions and container digests in lockfiles. Write docs/DEPENDENCIES.md with versions, licenses, support assumptions, and the date checked. Do not use unpinned latest tags in production.

5.2 Data flow

```
Phone browser
-> create scan and upload intent
-> private object upload
-> finalize upload / durable job acceptance

API + PostgreSQL transaction
-> scan state + job + outbox record
-> dispatcher -> broker -> recognition worker

Worker
-> validate and normalize image
-> detect regions and finalize their identities
-> crop each physical observation
-> retrieve artwork/full-card candidates
-> OCR and metadata checks
-> calibrated exact-match decision
-> review queue OR transactional inventory addition

Browser
-> authenticated progress polling
-> partial results, review, corrections, collection
```

5.3 Deployment boundaries

The web frontend and API may share a host initially. Run the recognition worker as a separately restartable process/container. Keep the database, broker, object storage, and authentication secrets off the public network wherever possible.

Do not put heavy recognition solely inside a request handler, a short-lived hosting function, or FastAPI's in-process background task facility. FastAPI's own guidance distinguishes heavier multi-process background work from its lightweight background-task pattern. [\[S14\]](#)

Do not assume a WordPress/shared-hosting account can run this system. Select hosting based on Python workers, durable storage, memory limits, networking, and operational requirements. Container-compatible VPS, managed services, or a private test server are deployment options, not an owner-approved production choice.

5.4 Keep the first version simple

One recognition service, one authoritative database, one broker, and private object storage are sufficient starting boundaries. Do not add multiple vector databases, Kubernetes, a model-serving cluster, or an LLM orchestration platform without measured need.

6. Card catalog, reference images, and provider use

6.1 Ingestion contract

Maintain a local, versioned print-level catalog. Select the appropriate Scryfall bulk export after verifying current documentation. An identity-only or artwork-only export is not sufficient for exact-printing inventory.

Store at minimum: provider ID, identity ID when present, card names and printed names, set code, collector-number string, language, layout, faces, image references, supported finishes, relevant treatment/promo fields, paper availability, and source timestamps. Preserve the source JSON for forward compatibility. The official card-object documentation is the schema authority. [\[S04\]](#)

Never convert collector numbers to integers. Preserve suffixes, unusual characters, and original formatting. Use the provider's printing identifier as the external reference key; names are not unique inventory keys.

Treat identity, printing, and face as separate concepts. A double-faced card can have multiple searchable faces but is one physical inventory copy. Do not create an additional copy because the back face was recognized in a targeted close-up.

6.2 Import workflow

Fetch the current bulk manifest, record timestamps and checksums, download to staging, validate schema, and stream the import. Upsert by stable provider ID. Build or update reference-image indexes in staging, then switch the active snapshot atomically.

Do not delete previously referenced printings because an import is incomplete. Mark unavailable or changed records and retain referential integrity. Keep the last known-good snapshot usable when the provider or importer fails.

Separate reference-image retrieval from JSON import. A metadata export does not mean that all image bytes are embedded in it. Cache permitted reference images and compute features once per relevant image/model version rather than repeatedly during user scans.

6.3 API discipline

Use a centralized server-side client with the required application-identifying and content-negotiation headers. Verify the current provider rules. Indexed Scryfall guidance asks API clients to remain below 10 requests per second; start with a shared limit of 5 requests per second, honor 429 responses and Retry-After, and use backoff. [\[S05\]](#)

Do not fan out public API lookups for every region of every frame. Recognition queries the local catalog. Background synchronization handles provider changes. Verify separate image-host rules rather than assuming an API rate limit applies to every host.

6.4 Price handling

Display **estimated value**, the provider, currency, selected finish, and an “as of” timestamp. Scryfall describes prices as estimates/market values rather than guaranteed final transaction prices. Do not market this integration as a live executable quote feed. [\[S06\]](#)

Keep unknown values null, not zero. Do not treat missing foil prices as nonfoil prices. Do not merge currencies. Do not claim a condition-adjusted valuation unless the source actually supplies it.

A copy with unknown finish is excluded from an exact finish-specific total by default. Show priced-copy coverage, for example “12 of 15 copies have a usable estimate.” A subtotal must not masquerade as the complete collection value.

6.5 Pack hints

Represent a selected product as a versioned hint profile: main set, known bonus-sheet sets, language assumptions, and provenance. Always include an escape route to the full catalog. A known set must not force an unreadable or out-of-set card into a wrong printing. [\[S09\]](#)

Without a verified profile, use the selected set only as a soft ranking feature. Never use expected rarity distribution or pack count to invent an identification.

7. Recognition pipeline

7.1 Stage A: ingest and normalize

Validate size, file signature, decoded dimensions, and supported format. Normalize orientation and color representation. Remove EXIF/location metadata from retained normalized assets. Keep an immutable source-image version and a content checksum for provenance.

Use bounded-memory decoding in an isolated worker. Handle corrupt files, decompression bombs, oversized dimensions, and decoder failures as normal error paths, not process-wide crashes. Upload security must follow a layered approach rather than trusting the filename or browser MIME type. [\[S15\]](#)

7.2 Stage B: detect physical card regions

Start with a classical OpenCV quadrilateral baseline for separated rectangular cards. Evaluate a licensed, maintainable detector or segmentation model when contours are insufficient. The detector must work across dark tables, busy mats, white borders, borderless layouts, sleeves, mild rotation, and varied lighting.

Return ordered quadrilateral corners or a polygon, normalized source coordinates, objectness diagnostics, occlusion/clipping flags, and a region version. Perspective transforms can rectify a detected quadrilateral; they do not recover detail that was never captured or content hidden behind another card. [\[S16\]](#)

Assign stable `observation_id` values before any inventory commit. Freeze the region set for that processing revision. Suppress overlapping duplicate detections of the same physical region, not visually identical cards at different locations.

If more than 24 plausible card regions are found, block automatic additions for that scan and request smaller groups. Do not silently process the first 24 and report the scan as complete.

7.3 Stage C: quality and layout

Evaluate native crop detail, readable text regions, blur, glare, edge clipping, perspective, occlusion, and supported layout. Run orientation checks when necessary. Store quality features and reason codes, not only a single opaque “quality percentage.”

Use layout-specific regions. Conventional frame, retro frame, borderless, split, adventure, and double-faced layouts must not share one blindly fixed crop map.

7.4 Stage D: candidate retrieval

Build two useful retrieval paths: artwork-focused matching and full-card/frame matching. Start with a reproducible embedding baseline and compare it against a lightweight image-hash baseline. Generic embeddings can be a prototype, not a production-accuracy claim.

Retrieve an initial candidate union, such as the top 25 from each available path, then expand all plausible printings and related faces of strong identity/artwork matches. Tune the candidate budget using held-out retrieval recall. Persist both the returned candidates and the catalog/index versions used.

Do not truncate away likely same-art reprints before exact-printing evaluation. Always keep a no-valid-match possibility. Similarity scores are internal diagnostics; they are not UI probabilities.

7.5 Stage E: OCR

Read the card name and, where available, collector number, set code, and language mark. Use multiple bounded preprocessing variants for difficult crops, such as grayscale, contrast adjustment, and modest deskewing. Tesseract documents several image-quality factors and preprocessing considerations relevant to recognition. [\[S17\]](#)

Preserve original OCR strings and normalized forms. Treat substitutions such as 0/0, I/1, and punctuation changes as alternatives requiring corroboration. Never silently rewrite an uncertain footer into the identifier of the most convenient candidate.

Missing text is not contradictory text. Evidence that clearly conflicts with the artwork must trigger review. An unreadable footer should not become a perfect set/number match merely because the user selected a set.

Bound preprocessing attempts and OCR time per crop. Do not run a general-purpose multimodal model on every video frame. An optional future fallback must have separate privacy, cost, licensing, and accuracy evaluation.

7.6 Stage F: rank and verify

Combine measured features: image similarities and gaps, name agreement, footer agreement, layout consistency, language evidence, quality, supported-category flags, and a pack hint. Include explicit missingness indicators.

Train or fit a ranker using labeled observations and realistic negative candidates. Then calibrate the **selected exact-match outcome** on held-out observations. Do not publish a fixed weighted formula such as 45% artwork plus 20% name as a probability.

A ranker is permitted to choose a candidate for review when confidence is low. It is not permitted to fabricate a provider ID, interpolate card metadata, or infer certainty from a top-three softmax that omitted the correct answer.

7.7 Stage G: policy decision

Pass the immutable recognition result into the policy engine described in Section 8. The decision must include model/index/calibration versions, effective threshold, reason codes, and metadata completeness.

Only the inventory service can commit a collection addition. Recognition workers must not issue ad hoc quantity increments outside that service.

8. Confidence, eligibility, and the 92% rule

8.1 Meaning of confidence

Use these distinct concepts:

Field	Meaning	Used for automatic addition?
p_identity	Estimated probability that the selected card identity is correct for this observation.	Informational prerequisite.
p_exact	Estimated probability that the observation is the selected supported card printing and language.	Yes, after qualification and gates.
finish_status	Unknown, explicitly selected, or independently verified finish.	Completeness/strict-mode gate.
condition_status	Ungraded or user-assessed condition.	Never a recognition confidence.

p_exact must be calibrated on the joint exact-match outcome, not calculated by multiplying separately calibrated marginal scores. It cannot exceed p_identity. Reject inconsistent probability outputs rather than silently adjusting them.

Keep explicit reasons for incomplete inventory metadata. “Exact printing matched” must not imply “foil and condition verified.” Some physically distinct treatments may require additional catalog fields or user information even after a base provider record is selected.

8.2 Automatic-add policy

The normal user threshold starts at **0.92**, and the comparison is **strictly greater than**. Evaluate the stored, unrounded probability. A UI display of 92.0% is not sufficient to decide which side of the boundary the raw value is on.

Every following gate must also pass:

1. Auto-add is enabled for the deployment and the user, and this scan recorded consent to the policy snapshot.
2. The model, calibration, catalog, and supported use-case bucket are qualified and available.
3. Detection is valid, the region is finalized, and the crop passes the qualified quality policy.
4. The chosen identity/printing/language exists in the active catalog snapshot and p_exact > threshold.
5. There is no contradictory evidence, unresolved same-art printing ambiguity, unsupported treatment, or unresolved physical-duplicate association.
6. The observation has not already been committed, ignored, superseded, or changed by a reviewer.
7. Any explicitly chosen finish is allowed for that printing. Strict metadata mode additionally requires a resolved finish.

The reference policy in this handoff implements the boundary and eligibility behavior, not a trained recognizer. Before calibration exists, route results to review even when raw similarity looks impressive.

8.3 Unknown finish and condition

Default behavior allows an eligible printing to be auto-added with `finish = unknown` and `condition = ungraded`. Display those fields prominently and exclude unknown finish from finish-specific valuation.

An optional strict mode holds the item for review until finish is resolved. A user can deliberately select a finish default for a scan, but the system must label that source as user-selected and reject a finish that the catalog does not permit.

Do not classify “nonfoil” merely because the image has no obvious glare. Do not infer Near Mint from an unopened-product workflow.

8.4 Why 92% is not a promise of an error-free pack

As an illustrative calculation, 15 independent decisions each having a true correctness probability of exactly 0.92 would have 0.92^{15} , approximately **28.6%**, probability of all being correct. Their expected error count would be **1.2**. Independence is only an illustration; real photo errors can be correlated. The expected error count follows by summing individual error probabilities.

Actual accepted observations should usually score much higher than the minimum. The release gate therefore measures precision at the actual deployed policy, not merely whether a percentage passes 92. If that policy fails the release gate, disable auto-add for the failing category or obtain owner approval for a stricter threshold. Never inflate probabilities to preserve automatic-add coverage.

8.5 Review and unknown outcomes

At or below threshold: review. Missing qualified probability: review. Conflicting evidence: review. Unsupported layout or poor-quality crop: review or close-up. Clearly identified non-card objects: exclude with an explanation. An uncertain object that could be a card must not disappear as a confident exclusion.

Show useful candidate differences rather than arbitrary decimal precision. Scores for a small candidate list need not total 100%; the correct candidate may be outside the list.

9. Physical copies, rescans, and duplicate protection

9.1 Three different duplicate problems

Duplicate detection: two bounding boxes refer to one card in one image. Resolve spatially before committing.

Repeated physical printing: two cards at different positions have the same printing. Preserve both observations and both inventory copies.

Repeated request or image: a client, queue, or user submits the same operation again. Use idempotency and explicit rescan handling; do not blindly increment quantity.

9.2 Stable observation model

Mint an observation for each physical region in a finalized image revision. Additional OCR passes, worker retries, candidate changes, and targeted close-ups attach to the same observation. A close-up endpoint must require its target observation ID.

A full-image retake after some copies have already been committed is not automatically safe to merge. Version 1 should offer a clear **new physical batch** versus **replace/review existing batch** choice. Replacing requires reconciliation; it must not simply rerun addition logic against newly generated region IDs.

9.3 Image hashes

Within an in-progress upload/scan, the same request key returns the existing operation. A repeated file hash in another scan should trigger a user-facing warning and a link to the previous batch. Do not use hashes as proof that two physical cards or two purchases are the same object.

Hash checks must be scoped to the current owner; do not expose another user's image existence. Permit intentional new-copy recording only through an explicit operation, not a retry flag controlled by an untrusted worker.

9.4 Multi-photo overlap is deferred

Matching card identities across overlapping photos is not enough to deduplicate physical cards. A future implementation must use geometric registration and instance association, and ask when identical copies cannot be distinguished. Do not implement “same card ID means same physical card.”

10. Durable background jobs and state machines

10.1 Durability contract

The server returns accepted status only after the immutable image reference, scan record, job record, and dispatch intent are durable. The browser may then leave. Unfinished uploads do not have this guarantee.

Use a transactional outbox: insert the job and outbox row in one database transaction; a dispatcher publishes work; workers are idempotent. Database state is authoritative even when a broker message or notification is lost.

Queue task delivery is not the same as an exactly-once inventory effect. Celery documents the importance of idempotent tasks and acknowledgement choices. Make the database mutation safe to replay regardless of broker delivery behavior. [\[S18\]](#)

10.2 Scan processing states

```

DRAFT -> UPLOADING -> QUEUED -> PROCESSING -> PROCESSED
                |               |
                |               +-> PARTIAL_FAILURE
                +-----> FAILED
DRAFT / UPLOADING / QUEUED / PROCESSING -> CANCEL_REQUESTED -> CANCELLED

```

PROCESSED means recognition work ended, not that every card was added. Track a separate resolution state: PENDING, RESOLVED, or CLOSED_WITH_UNRESOLVED.

10.3 Observation states

```
DETECTED -> RECOGNIZING -> AUTO_ADDED
      -> NEEDS_REVIEW -> MANUALLY_ADDED
      -> NEEDS_CLOSEUP -> RECOGNIZING
      -> EXCLUDED
      -> FAILED

AUTO_ADDED / MANUALLY_ADDED -> CORRECTED or REMOVED
```

Store processing attempts separately from the current decision. Retrying recognition must not reset accepted manual work.

10.4 Leases, retries, and progress

Use leases/heartbeats for running work, bounded retry counts, exponential backoff with jitter, and a dead-letter/recoverable-failure path. Suggested initial maximum: three automatic attempts for transient failures; adjust after testing.

Transient examples: temporary storage failure, broker disconnect, upstream sync interruption. Permanent examples: corrupt image, unsupported format, invalid catalog identifier, explicitly excluded observation.

Version 1 uses authenticated progress polling with backoff. Poll every roughly two seconds while actively processing, then slow down when appropriate. Optional server-sent events may improve immediacy but must not be required for correctness. Reopening the app must reconstruct state entirely from the server.

10.5 Cancellation and partial results

Cancellation stops future work where possible; already committed copies remain until explicitly undone. Show that distinction. Check cancellation and the observation version immediately before committing. Batch undo must prevent still-running workers from adding more copies afterward.

One failed crop must not erase successful results for unrelated observations. A whole-image decode failure, however, cannot produce invented partial identifications.

11. Data model and integrity requirements

11.1 General conventions

Use UUID identifiers, UTC timestamps, explicit enums or constrained text, database migrations, and integer copy quantities. Store monetary estimates as decimal values with currencies, never binary floating-point money. Keep provider numbers that may contain letters as strings.

Every private row must be directly or transitively owned by a user. Derive the authenticated owner server-side; never trust a `user_id` in a request body. Apply ownership checks in every query, background operation, signed URL, export, and object lookup.

11.2 Required tables

Entity	Essential fields	Integrity rule
users	id, auth issuer/subject, settings, created_at	Unique issuer/subject.
binders	id, owner_id, name, archived_at	All moved lots belong to the same owner.
catalog_snapshots	id, source, source_timestamp, checksum, status	Atomic active-snapshot selection.
card_identities	id, provider_identity_id, names	Identity is not a printing.
card_printings	id, provider_id, identity_id, set_code, collector_number, language, source_json	Unique provider ID; preserve original metadata.
card_faces	id, printing_id, face_index, image_ref, layout	Unique printing/face index.
reference_features	face_id, model_version, crop_kind, vector, checksum	Versioned features; no mixed embedding spaces.
pack_profiles	id, version, allowed_hint_sets, source_notes	Hints retain their provenance.
scans	id, owner_id, binder_id, processing_state, resolution_state, policy_snapshot, expected_count, revision	One durable scan identity.
scan_images	id, scan_id, immutable_object_ref, sha256, width, height, capture_path, expires_at	Cannot be replaced after acceptance.
observations	id, scan_id, image_id, polygon, region_revision, state, version, current_attempt_id	Stable physical-observation identity.
recognition_attempts	id, observation_id, versions, quality, OCR, candidates, p_identity, p_exact, reasons	Immutable attempt record.
review_decisions	id, observation_id, reviewer_id, decision, expected_version, created_at	Auditable manual override.
inventory_lots	id, owner_id, binder_id, printing_id, finish, condition, quantity_remaining, source_observation_id, version	At most one original lot per source observation.
inventory_events	id, lot_id, scan_id, event_type, delta, before_json, after_json, idempotency_key	Append-only audit and replay protection.

Entity	Essential fields	Integrity rule
jobs	id, scan_id, kind, state, attempt_count, lease_until, error_code	Database-authoritative execution status.
outbox_events	id, aggregate_id, payload, published_at	Replayable dispatch intent.
idempotency_records	owner_id, route, key, request_hash, response_ref	Same key plus different request body is a conflict.
price_snapshots	printing_id, finish, source, currency, amount, source_time, fetched_at	Missing price stays null.
model_registry	model/index/calibration versions, qualified_buckets, evaluation_ref, enabled	Only explicitly qualified combinations auto-add.

Use extra tables as needed for OAuth sessions, notifications, data-deletion tasks, and CSV imports. Do not substitute opaque JSON for all relational integrity.

11.3 Inventory lot semantics

A scanned observation creates a lot of quantity one. A manual or CSV import can create a larger lot. Collection views aggregate lots by printing, language, finish, condition, and binder; users may expand them to inspect provenance.

Finish is an extensible normalized field. At minimum support unknown, nonfoil, foil, and etched where the provider permits them. Preserve provider-specific treatment detail separately; do not collapse every premium treatment into one generic foil SKU.

Condition values start with ungraded, NM, LP, MP, HP, and damaged, with a clear indication that user assessments are not professional grading. Store metadata origin, such as user-selected versus catalog-constrained.

11.4 Required constraints and transaction pattern

Implement an owner-scoped idempotency key, a uniqueness constraint on non-null source_observation_id, nonnegative remaining quantity, valid state transitions, and optimistic version checks for review/correction.

```

BEGIN
  lock scan and observation rows
  verify owner, scan not cancelled/undone, observation version
  verify policy result and catalog reference are still applicable
  return existing committed result when source observation already has a lot
  insert inventory lot with quantity 1
  append ADD_SCAN inventory event with unique operation key
  update observation state and version
  write notification/outbox event if needed
COMMIT

```

The lock order must be consistent across auto-add, review, cancel, and undo to reduce deadlocks. Retry serialization/deadlock errors safely with the same operation key.

11.5 Corrections, removal, and undo

A correction changes the classification of the existing lot, records before/after metadata, and increments its version. It does not add quantity. Manual decisions take precedence over stale worker attempts.

Batch undo marks the batch as no longer accepting additions and reverses remaining inventory effects associated with that batch. Preserve already-consumed/removed quantities and report conflicts rather than driving totals negative. Never subtract arbitrary matching copies from unrelated batches.

12. API contracts

12.1 Common rules

Use `/api/v1`, JSON responses, documented error codes, and generated OpenAPI. Authenticate every private endpoint. Require an Idempotency-Key for mutating operations that can be retried. Use `If-Match` or an explicit `expected_version` for edits with race potential.

Common errors include 401 unauthenticated, 403 unauthorized, 404 not found, 409 conflict/version mismatch, 413 file too large, 415 unsupported type, 422 invalid input, and 429 rate limited. Do not reveal another user's resource through detailed authorization errors.

12.2 Endpoint inventory

Method and path	Purpose	Important behavior
GET <code>/catalog/search</code>	Search public catalog.	Paginated; no private collection data.
GET <code>/catalog/printings/{id}</code>	Retrieve printing and faces.	Return snapshot/source provenance.
POST <code>/scans</code>	Create scan with policy and binder choice.	Return scan ID and revision.
POST <code>/scans/{id}/uploads</code>	Create bounded upload intent.	Owner-bound temporary key and expiry.
POST <code>/scans/{id}/finalize</code>	Validate upload reference and accept work.	202 only after durable job/outbox write.
GET <code>/scans/{id}</code>	Retrieve current state and summary.	Safe after refresh or reconnect.
GET <code>/scans/{id}/observations</code>	Retrieve region results.	Paginated if needed; include versions.
POST <code>/scans/{id}/regions</code>	Add a missed region.	Reject duplicate spatial associations; new revision.
PATCH <code>/observations/{id}/region</code>	Correct a crop before or through review.	No silent re-creation of already-added copies.
POST <code>/observations/{id}/decision</code>	Accept candidate, choose another, or exclude.	Expected version and transactional inventory effect.
POST <code>/observations/{id}/closeups</code>	Add a targeted close-up.	Same observation, never a second copy.
POST <code>/scans/{id}/retry</code>	Retry eligible failed work.	Preserve accepted and manually resolved items.
POST <code>/scans/{id}/cancel</code>	Stop future processing/commits.	Existing additions remain visible.
POST <code>/scans/{id}/undo</code>	Reverse remaining batch inventory effects.	Atomic state change plus audited lot effects.

Method and path	Purpose	Important behavior
DELETE /scans/{id}/images	Delete photos/crops, retain chosen inventory.	Cancel image-dependent work as needed.
GET /collection	Search holdings.	Owner-scoped aggregation and pagination.
POST /collection/lots	Manually add a lot.	Explicit quantity and metadata.
PATCH /collection/lots/{id}	Correct, move, or adjust a lot.	Versioned audit event.
GET/POST /binders	List/create binders.	Owner-scoped.
PATCH /binders/{id}	Rename/archive binder.	No cross-owner movement.
POST /imports/csv	Validate CSV into a preview import.	No writes until confirmation.
POST /imports/{id}/commit	Commit reviewed rows.	Idempotent row effects.
POST /exports	Prepare owner-scoped CSV export.	Expiring private download.
GET /jobs/{id}	Retrieve owned import/export task state.	No broker details exposed.
GET/PATCH /settings	Get/change scan preferences.	Threshold and privacy changes audited.
DELETE /account	Request authenticated account deletion.	Reauthentication and deletion lifecycle.

12.3 Create-scan request example

This is a schema illustration, not a real product, pricing, or catalog record.

```
{
  "binder_id": "11111111-1111-4111-8111-111111111111",
  "mode": "pack",
  "expected_count": 15,
  "set_hint": null,
  "pack_profile_id": null,
  "auto_add_requested": true,
  "threshold": "0.92",
  "finish_default": "unknown",
  "condition_default": "ungraded",
  "require_known_finish": false
}
```

The server snapshots and enforces allowed settings. A client cannot bypass a disabled model registry, lower the approved threshold, or declare its own confidence.

12.4 Observation response example

All identifiers and measurements below are illustrative fixtures.

```
{
  "observation_id": "22222222-2222-4222-8222-222222222222",
  "version": 3,
  "region_index": 8,
  "state": "NEEDS_REVIEW",
  "polygon_normalized": [[0.10, 0.10], [0.20, 0.10],
                        [0.20, 0.30], [0.10, 0.30]],
  "selected_printing_id": "33333333-3333-4333-8333-333333333333",
  "p_identity": "0.995",
  "p_exact": "0.874",
  "reason_codes": ["PRINTING_AMBIGUOUS", "FOOTER_UNREADABLE"],
  "finish": "unknown",
  "condition": "ungraded",
  "inventory_lot_id": null,
  "versions": {
    "catalog": "fixture-catalog-1",
    "recognizer": "fixture-model-1",
    "calibration": "fixture-calibration-1"
  }
}
```

12.5 Error shape

```
{
  "error": {
    "code": "OBSERVATION_VERSION_CONFLICT",
    "message": "This card changed since you opened it. Refresh its result.",
    "retryable": false,
    "request_id": "44444444-4444-4444-8444-444444444444"
  }
}
```

Generate a frontend API client from the backend OpenAPI contract. Keep examples, types, and tests synchronized. Avoid separate handwritten interpretations of the confidence policy in the browser and worker.

13. Collection import, export, and interoperability

13.1 Canonical CSV

Support at least: quantity, scryfall_id, name, set_code, collector_number, language, finish, condition, binder, and notes. Export stable identifiers alongside readable names.

Imports must preview resolved rows, ambiguous rows, rejected rows, and quantity changes before commit. Do not choose the first same-name printing silently. Row-level idempotency must prevent duplicate effects after a retry.

An unknown printing may remain an unresolved import item, not a fabricated exact-printing lot. Preserve original row data so the user can repair it.

13.2 Safety and completeness

Handle UTF-8, quoted commas, newlines, BOMs, and names with punctuation. Prevent spreadsheet-formula injection in exported user-controlled cells using an explicitly documented escaping strategy. Preserve null prices and unknown finishes as unknown.

Do not claim ManaBox-format compatibility without testing against a current user-authorized export and documenting its mapping. A canonical CSV export is the Version 1 requirement; format-specific adapters are separate tests.

14. Dataset, model evaluation, and calibration

14.1 Required real-photo dataset

Build a consented dataset of real tabletop photos, not just clean reference images pasted into a grid. A suggested initial collection target is at least **600 source photos** across devices, layouts, lighting, card counts, and difficult cases. This is a planning target, not proof of statistical sufficiency.

Include 1-6, 7-10, 11-15, 16-20, and 21-24 count buckets. Include repeated physical printings, same-art reprints, full-art lands, sleeves, glare, rotated cards, partial crops, empty tables, other games, token/ad inserts, unusual treatments, non-English cards, and unsupported examples.

Store verified annotations: polygons, physical-instance IDs, identity, printing/language, visible face, finish when independently established, occlusion/quality, and whether exact printing is actually resolvable from that image. Unresolvable examples must remain ambiguous in evaluation.

14.2 Split discipline

Create separate training, calibration, and final test partitions. Split by capture session and physical-card group; do not scatter near-identical crops or augmentations across partitions. Include device and set/treatment stress partitions.

Reference-catalog images are the retrieval gallery, not a replacement for test photographs. Clearly distinguish gallery access from fitting on test observations. Hold back some identities from ranker training while leaving their reference images available for retrieval, and separately test items absent from the catalog.

Do not tune the threshold repeatedly on the final test set. Record dataset hashes, annotation revisions, random seeds, code version, and exclusion rules. Reviewer corrections used for training require consent and an ingestion review; they are not automatically trusted labels.

14.3 Metrics

Report detection precision/recall at a documented polygon-overlap criterion; duplicate-region rate; missed-card count per photo; retrieval recall at K; identity accuracy; exact-printing/language accuracy; auto-add precision; auto-add coverage; review rate; unknown-card rejection behavior; finish completeness; processing latency; and cost per accepted scan.

Calibration reporting must include reliability bins and sample counts, Brier score or an equivalent proper scoring metric, and observed error among auto-added results. Report per device/capture path/count bucket and important difficult categories. A pooled average cannot conceal a failing baseline phone.

Use uncertainty intervals. Because observations in one photo can share blur, glare, and layout conditions, report photo/session-clustered uncertainty rather than treating every crop as fully independent. Small segments are “insufficient evidence,” not automatically passed.

14.4 Proposed release gates

These are owner-approval targets, not achieved results.

Area	Initial gate	Consequence of failure
Auto-add correctness	At least 99.5% observed exact-printing/ language precision, plus a one-sided 95% lower bound of at least 99.0% under the deployed policy.	Keep category manual-only or revise policy with approval.
Evidence volume	At least 1,000 auto-added test observations overall, with adequate device/count/category representation and reported uncertainty.	Collect more evidence; no broad claim.
Qualified subgroups	No failing subgroup hidden by the pooled metric.	Disable auto-add for that subgroup.
Pack usability	At least 80% auto-add coverage on clean qualified 15-card photos, measured over all annotated card observations, not only detected ones.	Improve pipeline; do not relax correctness to hide review burden.
Detection	At least 98% recall on fully visible supported cards in the primary test subset, plus reported false positives and count errors.	Improve detection/recovery before broad release.
Inventory integrity	No duplicate side effects in retry/race tests; no silent loss or negative quantities.	Block release.
Unknowns	Unsupported/absent-catalog examples do not pass auto-add in the acceptance suite.	Block affected policy/model.
Privacy/security	No cross-owner data access in the test suite; retention/deletion pass.	Block release.

Coverage and precision must be reported together. A scanner that auto-adds almost nothing can have high precision without delivering the promised workflow.

The interval criterion and sample-size target are independent requirements. Meeting a round sample count does not automatically meet a confidence bound. Use exact-binomial or Wilson calculations as a secondary check and cluster-aware analysis for the release report.

14.5 Threshold qualification

Test the owner's requested threshold of 0.92 as written. Most qualifying predictions may lie far above it. When the high-confidence region is not well calibrated or has inadequate samples, keep auto-add off even if a plot looks favorable.

Version the calibration alongside the ranker, detector assumptions, image preprocessing, reference index, and supported buckets. A new model, OCR engine, or preprocessing change requires regression evaluation and may require recalibration.

15. Acceptance test matrix

Implement these tests as automated tests where possible and documented physical-device tests where necessary. Fixture probabilities are for policy testing only; they do not establish recognition accuracy.

ID	Scenario	Required result
A01	Qualified result with $p_{\text{exact}} = 0.93$, all gates true.	Exactly one auto-added copy.
A02	$p_{\text{exact}} = 0.92$ exactly.	Review.
A03	$p_{\text{exact}} = 0.9199$.	Review.
A04	High raw similarity without a qualified calibration.	Review; no percentage misrepresented as probability.
A05	Identity certain, same-art printing ambiguous.	Printing review.
A06	OCR set/number contradict artwork.	Review, regardless of high score.
A07	Unknown finish in normal mode.	Eligible printing may add; finish remains unknown and valuation incomplete.
A08	Unknown finish in strict mode.	Review.
A09	User chooses a finish absent from allowed metadata.	Review/validation error, not forced conversion.
A10	Two physical copies of the same printing in one image.	Quantity increases by two.
A11	Two detection boxes describe one physical card.	One observation after reconciliation; one addition.
A12	Worker crashes after commit but before acknowledgement.	Retry returns existing effect; quantity unchanged.
A13	User taps confirmation twice.	One effect and consistent response.
A14	Reviewer and worker race.	One winning version; manual decision is not overwritten by stale work.
A15	Close-up resolves an already tracked observation.	Existing copy updated; no extra copy.
A16	Same file is uploaded as an accidental new batch.	Warning/reconciliation; no silent repeat addition.
A17	User explicitly records genuinely new copies.	Separate audited batch allowed.
A18	Main-set hint with a bonus-sheet card.	Correct candidate remains reachable.
A19	Expected count differs from detected count.	Persistent mismatch warning; missing cards not invented.
A20	A card is missed by detection.	User can add its region and process it once.
A21	Glare obscures the footer.	Name may be suggested; exact printing reviewed when needed.
A22	More than 24 plausible regions.	Explain overflow and request split; no silent first-24 completion.
A23	Ad, generic card back, or another game's card.	Exclude when certain; otherwise unknown/review; never forced MTG match.

ID	Scenario	Required result
A24	Double-faced card shown from either face.	Correct physical printing candidate; one copy per observation.
A25	Non-English/unsupported category not qualified.	Review rather than inheriting English qualification.
A26	Browser closes before upload completes.	No claim of durable processing; resume/retry path on return.
A27	Browser closes after accepted upload.	Server completes; results available on return.
A28	Queue publication fails after database commit.	Outbox retries; job is not lost.
A29	One crop fails during a pack scan.	Other valid observations remain processed and visible.
A30	Cancel races with a pending addition.	Scan lock/version prevents post-cancel commits.
A31	Undo races with a worker retry.	No additions after undo; prior unrelated holdings preserved.
A32	Lot was already partly removed before batch undo.	Undo remaining effect only; no negative quantity.
A33	A user's ID is substituted in image/result/export requests.	No cross-owner access.
A34	Corrupt file, MIME mismatch, huge decoded image, expired upload.	Bounded failure with no unsafe processing.
A35	User reuses an idempotency key for a different body.	Conflict, not a second operation.
A36	A signed temporary upload is overwritten after finalization.	Immutable accepted object version is unaffected.
A37	Provider/catalog sync fails.	Last good snapshot remains active; stale status shown.
A38	Price missing or finish unknown.	Unknown value and coverage count; not zero or false live value.
A39	CSV retry, unknown printing, formula-like notes.	No duplicate import; ambiguous preview; safe export.
A40	Retention expires or user deletes images.	Photos/crops removed; retained batch metadata remains understandable.
A41	Account deletion with queued jobs.	Jobs cannot recreate deleted data; storage cleanup verified.
A42	Probabilities are null, out of range, NaN, or exact exceeds identity.	Review/fail closed with logged validation reason.
A43	Phone returns only a small video-frame capture.	Actual dimensions recorded; quality gate can request fewer cards.
A44	Catalog/model changes during processing.	Pinned scan versions or explicit controlled restart; no mixed-version decision.
A45	Keyboard/screen-reader review and expired photo.	Usable navigation, clear state, no inaccessible color-only result.

16. Security, privacy, and rights

16.1 Authentication and access control

Use a maintained identity provider. Validate issuer, audience, signature, expiry, and session state at the trusted backend. Protect cookie-authenticated mutations against CSRF. Use secure, HttpOnly cookies where applicable; configure CORS narrowly. Never accept a browser-supplied role or owner ID as authority.

Require elevated authorization for model activation, production feature flags, retention settings, and catalog imports. Avoid routine staff access to user photographs; any support access needs an auditable policy.

16.2 Upload and storage controls

Use private buckets and short-lived, owner-scoped upload/download authorizations. Generate object keys server-side. Bind upload limits and expected type where supported, and revalidate on the server.

Finalize an immutable object version. A reusable temporary signed upload must not permit the bytes of an accepted scan to change later. Either pin a storage version or copy verified bytes to a final server-only key and process that reference.

Use allowlisted external catalog/image hosts. Do not let user-provided image URLs become arbitrary server-side fetches. Keep raw images and OCR payloads out of normal logs. Security principles and file validation must be reviewed against OWASP's upload guidance. [\[S15\]](#)

16.3 Starting retention policy

Delete abandoned temporary uploads after **24 hours**. Retain normalized scan photos and close-up crops for **7 days** by default. Retain detailed OCR/candidate diagnostic payloads for **30 days**, then keep only the minimal chosen-result, version, and decision provenance needed for inventory history.

Users can delete images earlier without deleting accepted collection entries. Account deletion overrides normal retention and prevents workers from recreating data. Document backup expiration and deletion propagation separately; do not promise instantaneous erasure from all backups when the infrastructure cannot provide it.

Do not use user photos or corrections for model training without separate opt-in consent. Training storage must be separated from operational retention and support consent withdrawal procedures.

16.4 Intellectual property and commercial use

Use original UI and branding. Do not copy ManaBox source code, proprietary models, artwork packaging, or interface assets. Do not claim affiliation with Wizards, Scryfall, or ManaBox.

Before public release, verify data-source terms, image use, reference-image storage, model/dataset licenses, attribution, and trademarks. Scryfall's indexed API terms restrict paywalling its data; Wizards' Fan Content Policy also contains access and commercialization conditions. These policies are not blanket permission for every paid application or training use. Obtain appropriate review before monetization or claims of authorization. [\[S19\]](#)[\[S20\]](#)

Keep a source/license register covering code libraries, pretrained weights, reference images, and training photographs. A software library's license does not automatically settle the rights to model weights or artwork used with it.

17. Performance and cost budgets

17.1 Initial service objectives

Use the following as implementation targets to benchmark and revise, not as published promises: after an accepted upload, first useful results within a median of approximately 5 seconds; a qualified 15-card job within a median of approximately 15 seconds and a 95th percentile of approximately 30 seconds on the documented test configuration.

Report upload time separately. Also report queue wait, decode/detection, retrieval, OCR, policy, and commit time. Test concurrency levels of 1, 5, and 10 scans and record hardware, worker count, memory, and CPU/GPU use. If targets fail, report measured results and bottlenecks rather than disguising queue time.

17.2 Starting resource experiment

Begin with a documented CPU-only worker experiment, for example 4 vCPU and 8 GB RAM, and measure. This is not a guaranteed production size. Load models once per worker, bound concurrent decodes/OCR tasks, and avoid multiplying model memory by uncontrolled process counts.

Add GPU inference only when measured throughput/cost requires it. Keep CPU and GPU model outputs regression-tested. The project must run without a paid general-purpose AI call per card in the baseline design.

17.3 Cost model

Track cost per scan, per detected card, and per correctly auto-added copy. Include compute, object storage, requests, network transfer, database, authentication, monitoring, and support. Image bytes and retention often matter as much as inference time.

```
approximate worker cost per scan
= billed worker time allocated to scans * worker hourly cost / 3600

approximate photo storage
= scans per day * average retained bytes * retention days

cost per correctly auto-added copy
= attributable operating cost / correctly auto-added copies
```

Do not hard-code current hosting prices into the application. Obtain current quotes when selecting deployment, and keep operational estimates separate from performance claims.

17.4 Backpressure

Limit concurrent jobs per user and per worker. Start with two active processing jobs per account and a small bounded queue, configurable by deployment. Return a clear capacity message when limits are reached. Rate-limit upload intents, search, imports, and expensive retries.

18. Observability, administration, and recovery

Use structured request/job/scan/observation IDs. Monitor accepted-upload failures, queue depth, oldest-job age, worker heartbeats, processing-stage latency, failed crops, review reasons, user corrections, auto-add rate, catalog freshness, and image-retention backlog.

Break recognition metrics down by model version, calibrated bucket, and capture path. Changes in correction rate or review reasons should trigger investigation; they do not prove the true error rate because users may not inspect every result. Periodic audited samples are necessary.

Provide an admin view for failed jobs, last catalog import, active model/calibration pairs, qualification reports, and global/per-bucket auto-add kill switches. A kill switch must stop future automatic commits without destroying queued observations or accepted inventory.

Persist backups of the database and required configuration. Test restoration. Version model artifacts and keep the previous validated deployment available. Do not change database schemas in a way that prevents rollback without a documented migration plan.

Required runbooks: stuck queue; storage failure; catalog unavailable; model regression; duplicate-add incident; image-deletion failure; credential rotation; backup restore; and disabling auto-add while preserving manual review.

19. Repository and developer deliverables

19.1 Target repository structure

The implementation developer must create these components. The handoff package itself contains instructions and reference policy assets, not these application services.

```
mtg-pack-scanner/
  apps/web/           # Next.js UI and PWA
  services/api/       # FastAPI, auth, catalog, inventory, OpenAPI
  services/worker/    # Detection, retrieval, OCR, calibration
  packages/contracts/ # Generated API client and shared enums
  packages/policy/    # Single source of decision rules
  migrations/         # Versioned database changes
  infra/              # Containers, deployment, reverse proxy
  scripts/            # Catalog import, index build, evaluation
  tests/unit/
  tests/integration/
  tests/e2e/
  tests/fixtures/     # Clearly labeled synthetic/test-only data
  evaluation/         # Dataset manifests and reproducible metrics
  docs/               # ADRs, setup, API, privacy, runbooks
  .env.example
  compose.yaml
  Makefile
```

19.2 Required commands

Implement and document `make setup`, `make dev`, `make migrate`, `make seed-demo`, `make import-catalog`, `make build-index`, `make test`, `make test-e2e`, `make evaluate`, `make backup`, and `make restore-test`, or equivalent clearly named commands.

`seed-demo` must create obviously labeled synthetic fixtures and must never be the production data path. `evaluate` must produce a reproducible report from an explicit dataset manifest, model version, and policy configuration.

19.3 Required environment settings

Provide examples for public application URL, database URL, broker URL, object-storage endpoint/bucket/region, authentication issuer/audience, catalog user-agent identity, catalog rate limit, image limits, retention, worker concurrency, deployment auto-add switch, and model/calibration/index locations.

No secret may be prefixed for public frontend exposure. Validate configuration at startup. A missing production auth or model setting must fail closed; do not silently activate development bypasses.

19.4 Completion package from the developer

Deliver source code; lockfiles; database migrations; container/deployment configuration; environment template; operating instructions; API specification; automated tests; dataset/evaluation manifests; measured device report; calibration report; license register; privacy/retention behavior; runbooks; backup/restore evidence; and a requirement-by-requirement completion checklist.

State exactly what is implemented, what is mocked, what has been measured, and what remains unsupported. A styled camera screen connected to placeholder recognitions is not a finished scanner.

20. Build sequence and milestone gates

M0. Validate foundations

Verify provider terms and source schemas. Select hosting/authentication. Record current dependency versions. Test real capture dimensions on the oldest target devices using both browser preview and native photo/upload paths. Finalize the dataset plan and agree on release metrics.

Exit evidence: architecture decisions, source/license register, device capture samples, and owner-approved definition of a supported scan. Do not begin by polishing a dashboard while the basic capture path is untested.

M1. Build the recognition experiment

Import a print-level reference subset. Detect separated cards in real photos. Implement crop normalization, artwork/full-card retrieval, name/footer OCR, candidate expansion, and an unknown outcome. Compare OCR-only, image-only, and hybrid baselines on the same photos.

Exit evidence: reproducible evaluation notebook or script, real sample results, error taxonomy, and retrieval/detection measurements. No automatic inventory writes yet.

M2. Build the end-to-end manual-review slice

Implement auth, private storage, scan creation/upload/finalization, durable jobs, partial result retrieval, review UI, one-copy inventory lots, binder views, targeted close-ups, and idempotency. Route every recognition to review while the calibration gate is closed.

Exit evidence: one real photo can create correct reviewed inventory, survive browser closure and worker retry, and be undone. Cross-owner access and duplicate-add tests pass.

M3. Qualify recognition and confidence

Expand real-photo data. Fit the ranker/calibrator, document supported categories, and test the 0.92 policy. Improve the detector/retriever where the error analysis requires it. Evaluate exact printing separately from card-name success.

Exit evidence: locked evaluation report, uncertainty, supported-device/count buckets, and a model-registry entry. Unqualified categories remain review-only.

M4. Enable gated automatic addition

Wire the qualified policy to transactional inventory. Complete cancel/undo races, correction history, unknown-finish behavior, price coverage, CSV import/export, and the full acceptance suite.

Exit evidence: acceptance results and an end-to-end pack flow with no fake confidence values. Auto-add is enabled only for qualified buckets.

M5. Harden and release

Complete physical-device tests, accessibility, performance/load tests, backups/restores, retention/account deletion, secrets handling, monitoring, runbooks, and rollback. Publish only supported claims.

Exit evidence: deployment from a clean environment, production checklist, recovery rehearsal, and owner sign-off.

Phase 2. Extend after the baseline is proven

Potential additions: broader language qualification; 16-24-card auto-add qualification; better unusual-layout coverage; faster continuous single-card capture; geometric multi-photo sessions; optional push notifications; native camera integration; and deck-list export. Each extension has its own dataset and acceptance gate.

21. Risk register and practical mitigations

Risk	What can go wrong	Required mitigation
Tiny footer text	Correct name but wrong printing.	Per-printing uncertainty, close-up, no forced match.
Same artwork	Several printings are visually similar.	Candidate expansion and explicit ambiguity gate.
Capture limitations	Browser returns preview-sized pixels.	Inspect real dimensions; supported fallback and smaller layouts.
Foil/treatment ambiguity	Wrong SKU and valuation.	Unknown metadata or explicit user selection.
Missed card	User assumes the whole pack was counted.	Region overlay, count warning, add-region recovery.
Repeated physical copies	Dedup erases legitimate quantity.	Instance-based observations, not name/ID dedup.
Retried job	Collection quantity doubles.	Unique source observation and transactional idempotency.
New-set/index drift	Confidence degrades after updates.	Versioned snapshots, regression tests, per-bucket kill switch.
Account/image exposure	Photos or holdings leak.	Ownership checks, private objects, scoped URLs, security tests.
Rights restrictions	Business model conflicts with terms.	M0 review, license register, no unapproved monetization.
Too much review	Technically accurate but tedious.	Measure coverage and correction effort alongside precision.
Unrealistic scope	Team builds a general MTG platform first.	Milestone gates centered on real pack scanning.

22. Developer/coding-agent handoff prompt

Use the following prompt with this complete document and the accompanying reference assets:

Build the MTG Pack Scanner described in MTG_SCANNER_BUILD_INSTRUCTIONS.md. Treat its requirements, confidence semantics, data-integrity rules, and acceptance tests as authoritative.

First inspect the repository and current environment. Record architecture choices and current dependency versions. Work through M0-M5 in order, starting with real camera capture and a measured recognition experiment. Do not begin with a mock-only finished-looking frontend.

The product must process multiple separated MTG cards from one uploaded photo. The primary target is 15 cards on approximately 2022-and-newer phones, subject to actual device/photo qualification. Do not claim that this accuracy or card count has already been achieved.

Use hybrid image retrieval plus OCR. A raw OCR score, cosine similarity, or weighted average is not a calibrated exact-printing probability. Eligible results strictly above 0.92 may auto-add only after all policy and qualification gates pass. Keep automatic addition disabled until those gates are supported by real-photo evaluation.

Run durable recognition jobs on the server after upload acceptance. Preserve distinct physical copies. Retried jobs, repeated confirmation, and targeted close-ups must not duplicate inventory. Use stable observations, transactions, idempotency, and audited corrections/undo.

Keep finish unknown and condition ungraded unless explicitly established. Provide review for ambiguous printings, missing regions, glare, unknown cards, unsupported categories, and contradictions. Never fill a pack by inventing matches. Show timestamped estimated prices, not invented live quotes. Preserve null/unknown values.

Implement the actual API, database migrations, workers, private storage, review UI, collection, exports, privacy lifecycle, tests, and deployment instructions. Verify data-provider and model/image licenses. Label all fixtures and mocks. Do not use synthetic probabilities as accuracy proof.

For each milestone, report working features, commands to reproduce, tests run and their real outcomes, remaining limitations, and evidence for the next gate. Do not mark a feature complete because a screen exists. When a gate fails, preserve a working manual-review path and document the failure rather than silently weakening safety or accuracy requirements.

23. Definition of done and owner handoff

The owner can create an account, photograph a supported 15-card layout, leave after upload acceptance, return to completed or actionable partial results, review uncertain items, inspect exact-printing and unknown metadata, verify quantities, export the collection, and undo the batch without harming prior holdings.

The development team can reproduce the deployment, test suite, recognition benchmark, qualification report, catalog import, backup restore, and model rollback. The released claims match measured evidence. No unqualified category is silently auto-added.

Owner approval is still needed for: final brand/domain, hosting/provider spend, authentication provider, any change to the requested threshold, the supported launch-device matrix, the release metrics, and any monetization or use of customer photos for training.

Final instruction: favor a correct, recoverable record over a confident guess. The scanner's job is to reduce work, not make the user audit hidden mistakes.

24. Source register and verification notes

Sources were checked or sought on September 17, 2026. Official documentation and primary sources are used below. Documentation can change; verify current API behavior and terms again at implementation and release. Numerical product targets, proposed architecture, schema design, sample payloads, and release gates in this document are original specifications, not performance claims from these sources.

[S01] MDN: MediaDevices.getUserMedia(). Secure context and permissions.

<https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>

[S02] MDN: ImageCapture.takePhoto(). Still capture returns a Blob; requested settings need actual-output verification.

<https://developer.mozilla.org/en-US/docs/Web/API/ImageCapture/takePhoto>

[S03] MDN: Offline and background operation. Worker lifecycle and background-operation limitations.

https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Offline_and_background_operation

[S04] Scryfall: Card Objects. Primary schema reference; indexed extracts checked, direct page retrieval blocked during preparation. Verify complete current schema in M0.

<https://scryfall.com/docs/api/cards>

[S05] Scryfall: API access troubleshooting. Primary indexed guidance on identifying headers and API traffic limits; full current guidance must be verified in M0.

<https://scryfall.com/docs/faqs/i-m-having-trouble-accessing-the-scryfall-api-or-i-m-blocked-17>

[S06] Scryfall: public pricing disclaimer. Prices described as estimates/market values; consult stores for final prices. Primary-domain indexed extracts checked.

<https://scryfall.com/>

[S07] Guo et al.: On Calibration of Modern Neural Networks. Primary research on model confidence calibration. Abstract/record consulted, not a claim of MTG-specific performance.

<https://arxiv.org/abs/1706.04599>

[S08] scikit-learn: Probability calibration. Calibration concepts, methods, and evaluation.

<https://scikit-learn.org/stable/modules/calibration.html>

[S09] Wizards: Collecting Aetherdrift. Concrete example of main-set and Special Guests cards sharing a booster product.

<https://magic.wizards.com/en/news/feature/collecting-aetherdrift>

[S10] Apple: iPhone 14 technical specifications. Older-phone camera baseline example only, not web-capture output assurance.

<https://support.apple.com/en-us/111850>

[S11] MDN: HTML capture attribute. File-input camera capture behavior.

<https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/capture>

[S12] ONNX Runtime: Python. Inference runtime documentation.

<https://onnxruntime.ai/docs/get-started/with-python.html>

[S13] pgvector: official repository/documentation. PostgreSQL vector-search extension and indexing behavior.

<https://github.com/pgvector/pgvector>

[S14] FastAPI: Background Tasks. Guidance on lightweight background tasks versus heavier distributed work.

<https://fastapi.tiangolo.com/tutorial/background-tasks/>

[S15] OWASP: File Upload Cheat Sheet. Layered upload validation and storage security guidance.

https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[S16] OpenCV: Geometric Transformations of Images. Perspective-transformation tooling.

https://docs.opencv.org/4.x/dd/d52/tutorial_js_geometric_transformations.html

[S17] Tesseract: Improving the quality of the output. OCR preprocessing and image-quality considerations.

<https://tesseract-ocr.github.io/tessdoc/ImproveQuality.html>

[S18] Celery: Tasks. Idempotent task behavior, acknowledgements, and retries.

<https://docs.celeryq.dev/en/stable/userguide/tasks.html>

[S19] Scryfall: REST API documentation and usage terms. Primary indexed extract confirms data-paywall restrictions; direct page retrieval blocked. Full terms remain an explicit M0 verification item.

<https://scryfall.com/docs/api>

[S20] Wizards: Fan Content Policy. Access, attribution, and use conditions. A release-specific rights review remains necessary.

<https://company.wizards.com/en/legal/fancontentpolicy>

Required provider references not fully retrieved

Verify Scryfall's current bulk-data and image documentation directly before implementation. They are listed here as pending verification, not as inspected evidence:

<https://scryfall.com/docs/api/bulk-data>

<https://scryfall.com/docs/api/images>

25. Contents of this handoff package

The Markdown file is the authoritative editable specification. The PDF is a reading/printing copy of the same content. The reference configuration and policy module demonstrate fail-closed eligibility and the strict 92% boundary. Their unit tests verify policy logic only.

The package does **not** contain a trained detector, OCR model, artwork index, card catalog, frontend, deployed API, or measured device benchmark. Those are implementation deliverables listed above.

Files: MTG_SCANNER_BUILD_INSTRUCTIONS.md, MTG_SCANNER_BUILD_INSTRUCTIONS.pdf, assets/policy_reference.py, assets/config.example.json, tests/test_policy_reference.py, docs/ACCEPTANCE_CHECKLIST.md, docs/REFERENCE_TEST_RESULTS.txt, and README.md.